

DS in Java

Total points 26/40

The respondent's email (**maheshpatil9691@gmail.com**) was recorded on submission of this form.

✗ 3.The best technique for handling collision is *

0/1

- ☒ Separate chaining
- ☐ Double hashing
- ☐ Linear probing
- ☐ Quadratic probing

✗

Correct answer

- ☒ Quadratic probing

✓ 6.The number of comparisons done by sequential search is *

1/1

- ☐ $(N/2)-1$
- ☒ $(N+1)/2$
- ☐ $(N-1)/2$
- ☐ $(N+2)/2$

✓

✓ 36. What is the output of the above Java program?

1/1

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        int topElement = stack.peek();
        System.out.println("Top element: " + topElement);
        stack.pop();
        System.out.println("Stack after pop: " + stack);
    }
}
```

- ☒ A. Top element: 30, Stack after pop: [10, 20] ✓
- ☐ B. Top element: 10, Stack after pop: [20, 30]
- ☐ C. Top element: 30, Stack after pop: [30, 20]
- ☐ D. Top element: 20, Stack after pop: [30, 10]

✗ 21. Which of the following data structures is best suited for implementing an undo/redo functionality in a text editor?

* /1

- ☒ A) Stack ✗
- ☐ B) Queue
- ☐ C) Linked List
- ☐ D) Binary Search Tree

No correct answers

✗ 13. Which operation is used to remove the first node from a singly linked list? 0/1

- ☐ a) deleteFirst()
- ☐ b) removeFirst()
- ☐ c) deleteFront()
- ☒ d) removeFront()

✗

Correct answer

- ☒ b) removeFirst()

✓ 11. Which Java data structure allows you to store unique elements and provides constant-time performance for basic operations like add, remove, and contains? 1/1

- ☒ a) HashSet
- ☐ b) ArrayList
- ☐ c) LinkedList
- ☐ d) PriorityQueue

✓

✓ 24. Which of the following techniques can be used to resolve collisions in hashing? *1/1

- ☐ A. Rehashing
- ☐ B. Hash bucketing
- ☐ C. Linear probing
- ☒ D. All of the above

✓

✗ 15. To find the middle element of a singly linked list, you can use: *

0/1

- ☒ a) A loop that counts the elements and then finds the middle ✗
- ☐ b) The "Runner" technique with two pointers moving at different speeds
- ☐ c) Recursion to traverse the list and find the middle
- ☐ d) The "indexOf()" method in Java's LinkedList class

Correct answer

- ☒ b) The "Runner" technique with two pointers moving at different speeds

✓ 12. What will be the output of the following Java code snippet?

1/1

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> scores = new HashMap<>();
        scores.put("Alice", 85);
        scores.put("Bob", 90);
        scores.put("Charlie", 78);
        scores.put("David", 95);

        System.out.println(scores.containsKey("Charlie"));
    }
}
```

- ☒ a) true
- ☐ b) false
- ☐ c) Compile-time error
- ☐ d) NullPointerException



- ✓ 29. Which sorting algorithm is demonstrated in the following Java code snippet? *1/1

```
import java.util.Arrays;

public class SortingSnippet {
    public static void main(String[] args) {
        int[] arr = {7, 3, 5, 1, 9};
        Arrays.parallelSort(arr);
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

- ☐ A. Quick Sort
- ☐ B. Merge Sort
- ☐ C. Insertion Sort
- ☒ D. Arrays.parallelSort()



✓ 20. write the output. *

1/1

```
import java.util.LinkedList;

public class Question2 {
    public static void main(String[] args) {
        LinkedList<String> names = new LinkedList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");
        System.out.println(names.pollLast());
    }
}
```

- ☐ A) Alice
- ☐ B) Bob
- ☒ C) Charlie
- ☐ D) Compilation Error

✓

✓ 27. What does the studentScores.containsKey("Alice") method call return? 1/1

- ☒ A. true
- ☐ B. false
- ☐ C. NullPointerException
- ☐ D. Compile-time error

✓

✓ 37. What is the output of the above Java program? *

```
import java.util.HashSet;
import java.util.Set;

public class HashSetExample {
    public static void main(String[] args) {
        Set<Integer> numbers = new HashSet<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.remove(20);
        System.out.println("Numbers: " + numbers);
    }
}
```

- ☒ A. Numbers: [10, 30]
- ☐ B. Numbers: [20, 30]
- ☐ C. Numbers: [10, 20, 30]
- ☐ D. Numbers: [20]



✗ 33. Which of the following Java interfaces can be used to implement a stack? *0/1

- ☒ A. List
- ☐ B. Set
- ☐ C. Queue
- ☐ D. Deque

✗

Correct answer

- ☒ D. Deque

✓ 39. Which of the following statements is true about the get() method in a Java Map implementation? *1/1

- ☐ A. It adds a key-value pair to the map.
- ☐ B. It removes a key-value pair from the map.
- ☐ C. It updates the value of an existing key in the map.
- ☒ D. It retrieves the value associated with a specific key in the map.

✓

✓ 18. The process of inserting an element into a priority queue based on a certain order is called: *1/1

- ☐ a) Dequeue
- ☐ b) Poll
- ☒ c) Enqueue
- ☐ d) Heapify

✓

✓ 26. What type of keys does the "studentScores" HashMap hold in this code snippet? *1/1

- ☐ A. Integer
- ☒ B. String
- ☐ C. Double
- ☐ D. Character



✓ 40. What is the time complexity of the push(), pop(), and peek() methods in a Java Stack implementation using an ArrayList or LinkedList? *1/1

- ☒ A. $O(1)$
- ☐ B. $O(\log n)$
- ☐ C. $O(n)$
- ☐ D. $O(n^2)$



✓ 28. What is the output of the following Java code snippet? *

1/1

```
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

public static void main(String[] args) {
    int[] arr = {64, 34, 25, 12, 22, 11, 90};
    selectionSort(arr);
    for (int num : arr) {
        System.out.print(num + " ");
    }
}
```

- ☒ A. 11 12 22 25 34 64 90
- ☐ B. 90 64 34 25 22 12 11
- ☐ C. 11 12 22 34 25 64 90
- ☐ D. 90 11 12 22 25 34 64



✗ 12. What does the following Java code snippet do?

0/1

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(5);
        stack.push(10);
        stack.push(15);

        int topElement = stack.peek();
        System.out.println(topElement);
    }
}
```

- ☒ a) Removes the top element from the stack. ✗
- ☐ b) Prints the top element of the stack without removing it.
- ☐ c) Clears all elements from the stack.
- ☐ d) Throws an EmptyStackException.

Correct answer

- ☒ b) Prints the top element of the stack without removing it.

✗ 4. The depth of a complete binary tree is given by *

0/1

- ☐ $D_n = n \log_2 n$
- ☐ $D_n = n \log_2 n + 1$
- ☒ $D_n = \log_2 n$
- ☐ $D_n = \log_2 n + 1$

✗

Correct answer

- ☒ $D_n = \log_2 n + 1$

✓ 9. predict the output of the code *

1/1

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String args[]) {
        String a[] = { "A", "E", "I" };
        String b[] = { "O", "U" };
        List list = new ArrayList(Arrays.asList(a));
        list.addAll(Arrays.asList(b));
        Object[] c = list.toArray();
        System.out.println(Arrays.toString(c));
    }
}
```

- ☒ [A, E, I, O, U]
- ☐ [A, I, E, O, U]
- ☐ [U, A, E, O, I]
- ☐ [I, A, E, U, I]



✓ 34. When does a Queue underflow occur in Java? *

1/1

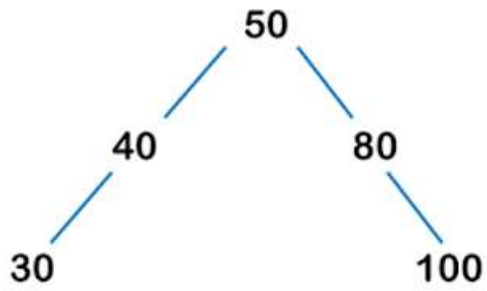
- ☒ A. When the Queue is empty and a poll() operation is performed.
- ☐ B. When the Queue is full and an offer() operation is performed.
- ☐ C. When the Queue contains only one element.
- ☐ D. When the Queue contains multiple elements.



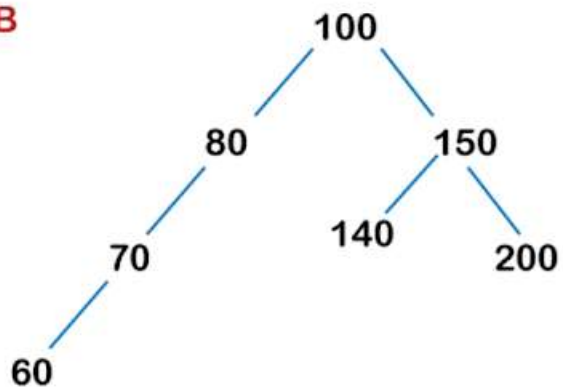
✓ 8. Identify the AVL tree among the following options? *

1/1

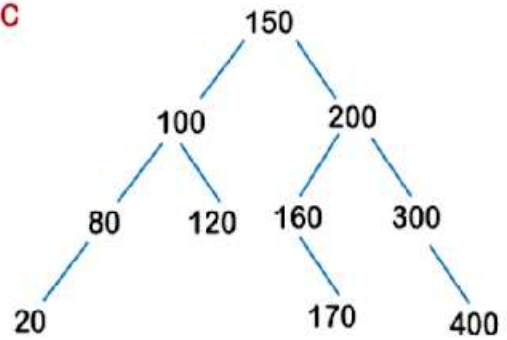
A



B



C



☐

A

☐

C

☒

Both A and C

☐

B



✗ 19. Which Java code segment adds an element "x" to the end of an ArrayList called "list"? *0/1

- ☐ a) list.addLast(x);
- ☐ b) list.insertLast(x);
- ☐ c) list.add(x);
- ☒ d) list.append(x);

✗

Correct answer

- ☒ c) list.add(x);

✗ 32. Which sorting algorithm is implemented in the following Java code snippet? 0/1

```
public class InsertionSort {  
    public static void insertionSort(int[] arr) {  
        int n = arr.length;  
        for (int i = 1; i < n; i++) {  
            int key = arr[i];  
            int j = i - 1;  
            while (j >= 0 && arr[j] > key) {  
                arr[j + 1] = arr[j];  
                j = j - 1;  
            }  
            arr[j + 1] = key;  
        }  
    }  
  
    public static void main(String[] args) {  
        int[] arr = {64, 34, 25, 12, 22, 11, 90};  
        insertionSort(arr);  
        System.out.println("Sorted array: " + Arrays.toString(arr));  
    }  
}
```

- ☒ A. Bubble Sort
- ☐ B. Merge Sort
- ☐ C. Quick Sort
- ☐ D. Insertion Sort

✗

Correct answer

- ☒ D. Insertion Sort

✗ 31. What is the main disadvantage of the Bubble Sort algorithm? *

0/1

- ☐ A. It is not stable.
- ☒ B. It is not suitable for large datasets.
- ☐ C. It has a slow time complexity.
- ☐ D. It requires additional memory.

✗

Correct answer

- ☒ C. It has a slow time complexity.

✗ 1. The time complexity used for inserting a node in a priority queue on the basis of key is: *

0/1

- ☐ O(n)
- ☐ O(n²)
- ☒ O(n log n)
- ☐ O(log n)

✗

Correct answer

- ☒ O(n)

✓ 5. The postfix form of A*B+C/D is *

1/1

- ☐ ABCD+/*
- ☒ AB*CD/+
- ☐ *AB/CD+
- ☐ A*BC+/D

✓

✓ 23. What is the time complexity of inserting an element into a hash table using a good hash function and handling collisions effectively? *1/1

- ☒ A. $O(1)$
- ☐ B. $O(\log n)$
- ☐ C. $O(n)$
- ☐ D. $O(n^2)$



✗ 2. Which of these will form an inversion in this given array? *

0/1

arr = {2,8,5,3}

- ☒ (2,8)
- ☐ (8,5),(8,3)
- ☐ (2,8),(2,5),(1,3)
- ☐ (8,5),(8,3),(5,3)



Correct answer

- ☒ (8,5),(8,3),(5,3)

✓ 16. What is the main advantage of a doubly linked list over a singly linked list? *1/1

- ☐ a) More memory-efficient
- ☒ b) Faster traversal from end to start
- ☐ c) Easier to implement
- ☐ d) Allows circular references



✓ 14. Which of the following statements is true about a circular linked list? * 1/1

- ☐ a) It cannot be traversed.
- ☐ b) It has no beginning or end.
- ☐ c) It cannot contain cycles.
- ☒ d) It has a tail node that points to the head node.



✓ 22. Which data structure in Java would you use to implement a "Last Recently Used" (LRU) cache? *1/1

- ☐ a) Stack
- ☐ b) Queue
- ☒ c) LinkedHashMap
- ☐ d) HashSet



✓ 7.If the array is already sorted, which of these algorithms will exhibit the best performance. *1/1

- ☐ A - Merge Sort
- ☒ B - Insertion Sort
- ☐ C - Quick Sort
- ☐ D - Heap Sort



✓ 30. Which of the following best describes the process of Bubble Sort? * 1/1

- ☐ A. It divides the list into halves and sorts each half separately.
- ☒ B. It compares adjacent elements and swaps them until the list is sorted. ✓
- ☐ C. It randomly rearranges the elements in the list until they are in order.
- ☐ D. It selects the maximum element and moves it to the front of the list.

✓ 17. In a singly linked list, what is the time complexity to insert an element at the end? *1/1

- ☐ a) $O(1)$
- ☐ b) $O(\log n)$
- ☒ c) $O(n)$ ✓
- ☐ d) $O(n^2)$

✗ 10. What is the time, space complexity of following code?

0/1

```
int a = 0, b = 0;
for (i = 0; i < N; i++)
{
    a = a + rand();
}
for (j = 0; j < M; j++)
{
    b = b + rand();
}
```

- ☒ A. $O(N * M)$ time, $O(1)$ space
- ☐ B. $O(N + M)$ time, $O(N + M)$ space
- ☐ C. $O(N + M)$ time, $O(1)$ space
- ☐ D. $O(N * M)$ time, $O(N + M)$ space

✗

Correct answer

- ☒ C. $O(N + M)$ time, $O(1)$ space

✓ 25. Read the code and answer the following (question 25-27)
What data structure is being used in the above Java code snippet?

1/1

```
import java.util.HashMap;

public class HashingSnippet {

    public static void main(String[] args) {

        HashMap<String, Integer> studentScores = new HashMap<>();

        studentScores.put("John", 85);
        studentScores.put("Alice", 92);
        studentScores.put("Bob", 78);
        studentScores.put("Eve", 96);
        studentScores.put("Mike", 70);

        System.out.println("Bob's score: " + studentScores.get("Bob"));
        System.out.println("Number of students: " + studentScores.size());
        System.out.println("Is Alice in the list? " + studentScores.containsKey("Alice"));
        System.out.println("Is the score 96 present? " + studentScores.containsValue(96));

        studentScores.remove("Mike");

        System.out.println("Updated number of students: " + studentScores.size());
    }
}
```

- ☐ A. ArrayList
- ☐ B. LinkedList
- ☒ C. HashMap
- ☐ D. TreeSet



✗ 35. What does the offer() method do in a Java Queue implementation? * 0/1

- ☐ A. Adds an element to the front of the Queue.
- ☐ B. Adds an element to the rear of the Queue.
- ☒ C. Removes an element from the front of the Queue. ✗
- ☐ D. Removes an element from the rear of the Queue.

Correct answer

- ☒ B. Adds an element to the rear of the Queue.

✓ 38. Which Java class provides a commonly used implementation of the Map interface? *1/1

- ☐ A. HashSet
- ☐ B. ArrayList
- ☒ C. HashMap ✓
- ☐ D. LinkedList

